Highlights

Pareto Memory Ranking Under a Fixed Candidate Set: Kernel Correctness and Microbenchmark Evidence

Anonymous authors

- A minimal Pareto ranker is inserted after fixed-candidate retrieval.
- Twelve targeted Java 21 tests pass with no failures or errors.
- A warmed 1,000-candidate, 1,024-dimensional kernel run has 2.497 ms median latency.
- No MSC or LongMemEval retrieval-quality claim is made.

Pareto Memory Ranking Under a Fixed Candidate Set: Kernel Correctness and Microbenchmark Evidence

Anonymous authors

Abstract

Long-horizon agents often combine several memory objectives in a single scalar score. Such a score is convenient, but it can hide trade-offs between a recently created memory and an older memory that is more relevant, important, or graph-connected. This paper evaluates a deliberately narrow intervention: inserting non-dominated sorting and crowding-distance selection into an existing memory-ranking path while leaving candidate generation and the memory-state update mechanism unchanged. The integrated adapter exposes four objectives—similarity, retention, importance, and graph relevance—whereas the node-only timing path measures the three objectives available without graph state. On the fixed test candidates, the Java 21 targeted suite ran 12 tests with zero failures and zero errors. A warmed $N = 1000$, 1024-dimensional node-ranking microbenchmark reported a median latency of 2.497 ms. These results support implementation-level correctness and a small-kernel timing claim only. They do not establish retrieval-quality improvement on MSC or LongMemEval, statistical significance, or production end-to-end performance; no frozen summary-to-session mapping was available for those claims.

Keywords: long-term memory, multi-objective ranking, Pareto frontier,

agent systems, reproducible microbenchmark

1. Introduction

Long-horizon agents need to decide which stored memories should enter a bounded context. Long-term memory systems have explored retrieval, consolidation, and memory reinforcement as separate mechanisms; representative examples include MemoryBank [1] and long-context conversation memory [2]. A benchmark such as LongMemEval makes the broader evaluation problem explicit, but it also requires an evidence mapping from a question to the sessions that should support an answer [3]. That mapping is not available in the experiment reported here.

The engineering question is therefore narrower: when the candidate set is already fixed, can a non-dominated ranker preserve objective trade-offs without changing candidate generation or memory-state updates? A weighted scalar score always imposes a single exchange rate between objectives. In contrast, non-dominated sorting keeps candidates that are incomparable across the objectives and uses crowding distance only when a front is larger than the requested output. The intervention is intentionally small: it changes ordering, not retrieval, storage, embedding, or answer generation.

This paper makes three bounded contributions. First, it describes the integration contract between the existing long-term memory path and a generic Pareto ranking kernel. Second, it records targeted tests that exercise strict dominance, incomparable trade-offs, crowding-distance boundary selection, invalid or empty inputs, graph-enabled integration, and no-graph fallback behavior. Third, it reports one warmed node-only microbenchmark on a fixed

candidate set. The four-objective adapter and the three-objective node-only timing path are kept separate so that the timing result is not misread as a measurement of graph computation or production retrieval.

The central claim is correspondingly modest: the integrated kernel is functionally testable and has a millisecond-scale timing observation under the stated fixed-candidate protocol. The paper makes no claim of higher MSC or LongMemEval quality, no significance claim, and no end-to-end production latency claim.

2. Related Work

MemoryBank treats long-term memory as an explicit component for retaining information across interactions [1]. Beyond Goldfish Memory studies long-term open-domain conversation and motivates mechanisms that avoid treating every past message as equally useful [2]. LongMemEval provides a benchmark framing for interactive memory, including the need to identify evidence across long histories [3]. These works motivate the problem setting, but their benchmark-level quality conclusions require data, labels, and evaluation mappings that are outside the present kernel-only study.

The ranking mechanism uses standard multi-objective ideas: Pareto dominance separates candidates that are uniformly no worse from candidates that trade one objective against another, and crowding distance favors boundary and less-dense points within a front. These operations are related to the non-dominated sorting and crowding-distance mechanisms popularized by NSGA-II [4]. Here they are used as a deterministic ranking primitive rather than as a population-based evolutionary optimizer. In particular, the cited

optimization literature does not by itself imply a memory-retrieval gain for this system.

3. Method

3.1. Scope and interface

Let $C = \{c_1, \ldots, c_n\}$ denote the candidate list returned by the existing memory path after its normal vector-store query and metadata scope filtering. The ranker receives C, a requested k, and objective vectors; it does not create new candidates, change embeddings, write to the vector store, or update reinforcement state. After ranking, the existing path continues with its retrieval-side reinforcement, so the selected objects may change even though the update mechanism itself is unchanged. This boundary makes the experiment a ranking-kernel study rather than an end-to-end retrieval study. Figure 1 summarizes the integrated path; it is an implementation overview, not an experimental result.

The long-term memory adapter constructs a four-dimensional objective vector

$$f(c) = \big(s(c),\ r(c),\ i(c),\ g(c)\big), \qquad f_j(c) \in [0, 1], \tag{1}$$

where s is calibrated similarity, r is dynamic retention, i is importance, and g is graph relevance. The adapter therefore retains the existing objective computations and delegates only the ordering decision to the Pareto kernel. The node-only benchmark intentionally excludes graph state: it evaluates the three-dimensional vector (r, s, i) available from a memory node, query embedding, and timestamp. Consequently, the reported latency is not a four-objective graph-path latency.

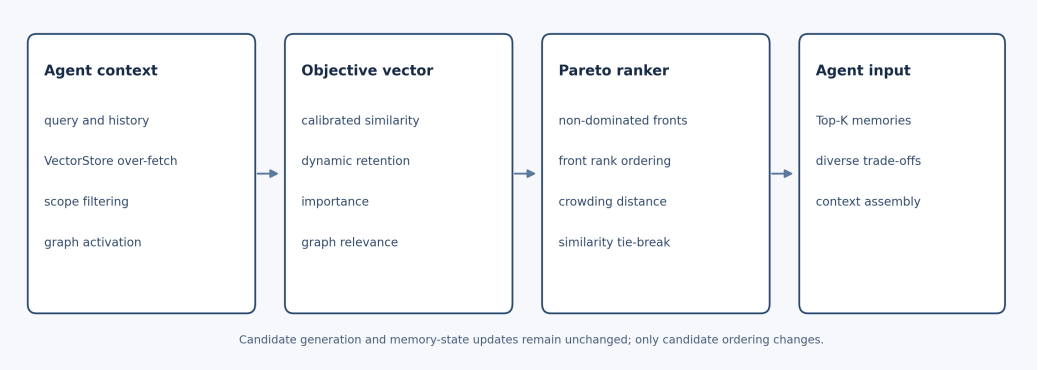


Figure 1: Conceptual implementation overview of the long-term memory ranking path. Candidate generation, scope filtering, and graph activation precede construction of the four-objective vector; Pareto ordering selects the Top-K candidates. Downstream context assembly and agent task success are not evaluated in this study.

3.2. Dominance and front construction

For two candidates a and b, a dominates b when

$$f_j(a) \geq f_j(b) \quad \forall j, \qquad \text{and} \qquad f_{j^*}(a) > f_{j^*}(b) \quad \text{for at least one } j^*. \tag{2}$$

Pairwise comparisons produce the first non-dominated front. Removing that front and repeating the comparison produces subsequent fronts. Front rank is therefore a partial-order signal: a candidate on a higher-priority front is not necessarily the best point on every individual objective, but it is not dominated by a candidate on a lower-priority front.

When a front contains more than k candidates, the kernel computes normalized crowding distance independently along each objective, assigns infinite distance to objective boundaries, and selects the largest distances before using relevance as the deterministic tie-breaker. If the first front is smaller than k, complete fronts are appended in rank order and only the final partial

front is truncated by crowding distance. This preserves extremes and avoids turning a trade-off into an arbitrary scalar exchange rate.

3.3. Numerical and boundary semantics

Objective values are normalized into the unit interval before comparison. Null or empty candidate lists and non-positive k return an empty result. Objective vectors with inconsistent dimensions are rejected by the generic API; node-level vectors with an incompatible embedding length are skipped by the node adapter. These checks are part of the kernel contract, not additional benchmark assumptions.

The generic implementation uses pairwise dominance comparisons and per-objective sorting inside each selected front. With d objectives, that path has dominant cost $O(n^2 d)$ for front construction, followed by $O(dn \log n)$ sorting work for crowding distance. The three-objective node adapter uses a discrete Fenwick-prefix fast path for first-front construction, reducing that step to $O(n \log n)$ before crowding-distance selection; the four-objective document adapter retains the generic path. The implementation keeps the candidate objects and their metadata intact; only the returned order changes.

4. Experimental Protocol

The protocol freezes the candidate set at the call boundary and evaluates the kernel in the Java 21 test environment used by the project. The targeted suite covers the ranker, dynamic-retention helper, and long-term-memory integration test classes. The ranker tests check dominance, trade-off fronts, crowding extremes, empty and invalid inputs, and the node-only timing path. The retention tests exercise the existing helper. The integration

tests check long-term-memory ranking, content-hash idempotency, and graceful operation when graph activation is absent. These tests are diagnostic and contract-oriented; they are not a replacement for a benchmark with frozen retrieval labels.

For the timing observation, the node-only ranker receives a fixed list of $N = 1000$ candidates with 1024-dimensional normalized vectors and returns a bounded top-k list. The JUnit measurement is warmed before the recorded samples and reports the median elapsed time. The result is reported as an observation on one machine and one runtime configuration, not as a production percentile or a hardware-independent guarantee.

No MSC or LongMemEval batch was executed as part of this protocol. In particular, no frozen mapping from a generated memory summary to source session identifiers was available. We therefore report neither retrieval precision/recall nor answer quality, and we do not perform significance tests.

5. Results

5.1. Contract-level correctness

The targeted suite completed all 12 tests with zero failures and zero errors. The result confirms that the exercised contracts execute successfully in the recorded Java 21 environment. It does not establish that every production integration or every candidate distribution is covered.

The functional tests include a strict-dominance case in which a uniformly better vector is assigned to an earlier front, and an incomparable case in which an old but relevant candidate and a fresh but less relevant candidate remain on the same front. A crowding-distance case retains objective-boundary

Table 1: Grounded evidence boundary for the reported study.

Artifact or path	Observation	Interpretation
Targeted Java 21 suite	12 tests; 0 failures; 0 errors	Exercised kernel, retention, integration, and fallback contracts pass.
Node-only timing path	$N = 1000$; 1024 dimensions; 2.497 ms median	Warmed microbenchmark observation for the three-objective node ranker.
External memory benchmarks	No frozen summary-to-session mapping	No MSC or LongMemEval quality result is identifiable from this study.

candidates when the front is truncated. Null, empty, and dimension-mismatch inputs exercise fail-safe behavior. The long-term-memory integration test also keeps both sides of an explicit freshness–relevance trade-off, while the no-graph constructor returns a usable result. These observations support the intended partial-order semantics and graceful fallback behavior, not a claim that the ranker is universally superior to a scalar baseline.

5.2. Node-only microbenchmark

The warmed node-only path reports a median latency of 2.497 ms for $N = 1000$ and 1024-dimensional candidates. The measured path computes

three objective values per node, performs non-dominated comparisons, and applies crowding distance when required. It does not include external vector-store latency, graph activation, embedding generation, network transfer, persistence, or model generation. The number is therefore useful as a kernel-scale reference point and not as an end-to-end service budget.

5.3. What the evidence does not show

Because the experiment freezes candidates and stops at ranking, it cannot distinguish a ranking improvement from an upstream candidate-generation effect. It also cannot measure answer faithfulness, session-level evidence recall, or task success. The absence of a frozen summary-to-session mapping is a data-boundary limitation rather than a negative quality result.

6. Limitations

The evidence is intentionally narrow. First, the timing path is a warmed JUnit microbenchmark on one machine, so it should not be read as a p50/p95 production measurement. Second, the timing path uses 1024-dimensional node vectors and excludes graph activation; it is not a measurement of the four-objective adapter under a production embedding or vector-store workload. Third, the study has no frozen summary-to-session mapping and therefore cannot support MSC or LongMemEval retrieval-quality claims, answer-quality claims, or statistical significance. Fourth, no quantitative comparison against a weighted-sum baseline was run under the same candidate fixtures. Fifth, the long-term-memory code path does not establish user-level isolation: the vector-store request supplies the query and top-k, after which scope filtering is applied; the user identifier is derived from returned candidates for

graph activation rather than applied as a vector-query filter. If users share a scope, this path does not by itself exclude cross-user candidates. The current tests therefore do not establish tenant isolation, and this security boundary requires separate remediation and verification before production safety can be claimed. The correct interpretation is feasibility and contract evidence, not superiority or isolation assurance.

7. Conclusion

This study isolates one engineering change: Pareto-frontier ordering after a fixed candidate set. The existing long-term-memory adapter can express four objectives, while the node-only benchmark measures the three-objective ranker that is independent of graph state. The targeted Java 21 suite passed 12 tests with no failures or errors, and the warmed $N = 1000$, 1024-dimensional kernel observation was 2.497 ms median. Together, these results support a small, testable ranking kernel under the stated protocol. They do not establish retrieval-quality gains, significance, or production end-to-end performance. A subsequent study would need a frozen summary-to-session mapping, a pre-registered candidate-generation protocol, a matched scalar baseline, and an independently controlled end-to-end evaluation before making those broader claims.

Funding

Funding information was not supplied with the study artifacts and must be confirmed by the authors before submission.

Declaration of competing interest

A competing-interest statement was not supplied with the study artifacts and must be confirmed by the authors before submission.

Data availability

The manuscript uses the repository-local kernel logs and metadata described in the evidence table. No external MSC or LongMemEval result file is used to support a quantitative claim in this paper.

Code availability

The ranking kernel and its targeted tests are part of the accompanying project worktree. The manuscript does not modify the production source as part of the paper-writing step.

CRediT authorship contribution statement

Author contributions were not supplied with the study artifacts and must be completed by the authors before submission.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript, an AI-assisted tool was used to help organize prose and check consistency against the repository-local evidence. This manuscript is a draft; the authors must review and edit the content as needed and take full responsibility before submission.

References

[1] W. Zhong, L. Guo, Q. Gao, H. Ye, Y. Wang, Memorybank: Enhancing large language models with long-term memory, arXiv preprint arXiv:2305.10250 (2023). doi:10.48550/arXiv.2305.10250.

[2] J. Xu, A. Szlam, J. Weston, Beyond goldfish memory: Long-term open-domain conversation, in: Proceedings of ACL, 2022, pp. 5180–5197. doi:10.18653/v1/2022.acl-long.356.

[3] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, D. Yu, Longmemeval: Benchmarking chat assistants on long-term interactive memory, arXiv preprint arXiv:2410.10813 (2024). doi:10.48550/arXiv.2410.10813.

[4] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: Nsga-ii, IEEE Transactions on Evolutionary Computation 6 (2) (2002) 182–197. doi:10.1109/4235.996017.

Appendix A. Supplementary Information

Appendix A.1. Reproducibility record

The reproducibility record consists of the project-local compilation log, targeted test log, and paper-data summary. The compilation log records the Java 21 release used for the backend build. The targeted log records the three test classes, their individual counts, the aggregate count, and the node-only timing samples. The final kernel log and paper-data summary provide the quantitative values reported in this manuscript.

Table A.2: Targeted test inventory recorded in the final kernel log.

Test class	Tests	Main contract
`LongTermMemoryEnhancedTest`	3	Integration, idempotency, and no-graph fallback
`ParetoMemoryRankerTest`	6	Dominance, trade-offs, crowding, boundaries, ties, and timing
`DynamicEbbinghausDecayTest`	3	Existing retention helper behavior
Aggregate	12	0 failures; 0 errors

Appendix A.2. Ranking pseudocode and invariants

The kernel can be summarized without reference to a dataset-specific score scale:

Algorithm 1 Pareto ranking and Top-K truncation

Require: Candidate vectors C, output limit K

Ensure: Ordered candidate list L with $|L| \leq K$

if $C = \emptyset$ or $K \leq 0$ **then**
  return [ ]
end if
Partition C into non-dominated fronts $F_1, F_2, \ldots$
$L \leftarrow [\,]$
for each front F_r in ascending rank order **do**
  if $|L| + |F_r| \leq K$ **then**
    Append F_r ordered by the deterministic tie-breaker
  else
    Assign normalized crowding distances within F_r
    Append the highest-distance candidates until $|L| = K$
    break
  end if
end for
return L

The invariants tested by the contract suite are: a strictly dominating candidate precedes a dominated candidate; incomparable candidates can share a front; boundary points receive priority under front truncation; empty and invalid inputs do not create synthetic memories; and the graph-disabled constructor still returns a result when vector retrieval supplies a candidate. The invariant set is deliberately stronger than a single example ranking but weaker than a quality guarantee on unseen conversations.

Appendix A.3. Evidence and interpretation boundary

Every quantitative statement in the manuscript points to either the paper-data summary or a kernel log. The four-objective description is an integration contract; the 1024-dimensional, $N = 1000$, 2.497 ms result is a node-only microbenchmark. The manuscript should therefore be read as an auditable kernel report, not as a benchmark paper for long-term-memory retrieval quality.